

Comparing Files

MODULE 2: FILE OPERATIONS, LINKS, AND DATA PROCESSING

INDEX

- Comparing Files
- cmp (Compare byte-by-byte)
- diff (Find line-by-line differences)
- sdiff (Side-by-Side difference)
- wdiff (Word-based differences)
- comm (Compare sort files)
- vimdiff (vim differential)

Comparing Files

In the world of Linux, managing and comparing files is a common task for system administrators and developers alike.

File comparison is the automated or manual evaluation of file contents, structure, or binary data to see how they diverge. Instead of reading through long files manually—which invites human error—Linux command-line utilities quickly isolate exact lines or bytes that changed.

Core Linux Comparison Tools :-

cmp
diff
sdiff
wdiff
comm

cmp (Compare byte-by-byte)

The **cmp** command is used for a quick check to see if two files are identical. It is highly efficient for large or binary files because it stops checking the moment it finds the very first difference.

Command: cmp file1.txt file2.txt

Output:

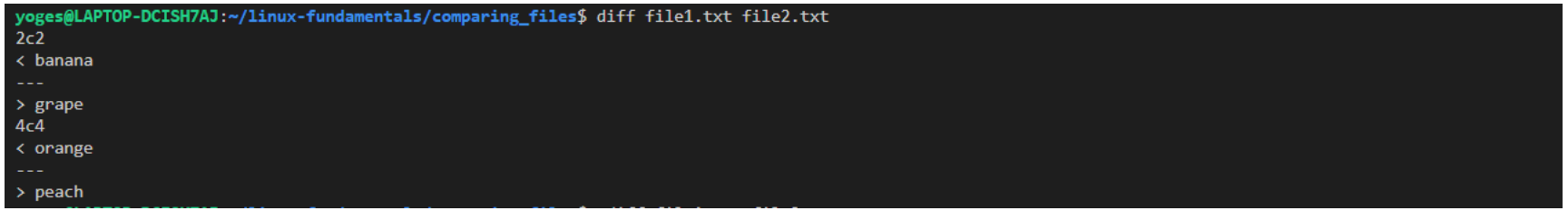
```
yoges@LAPTOP-DCISH7AJ:~/linux-fundamentals/comparing_files$ cmp file1.txt file2.txt
file1.txt file2.txt differ: byte 1, line 1
```

Takeaway: It tells you exactly **where** the first mismatch occurs (Line 2, Byte 7), but it does not show you what the actual differences are. If the files were identical, it would output nothing.

diff (Find line-by-line differences)

The diff command shows you exactly what changes need to be made to make the first file match the second file.

Command: diff file1.txt file2.txt
Output:



- Takeaway: The < symbol points to the contents of the first file, and > points to the second. The 2c2 means line 2 needs to be changed.
- 💡 Pro-tip: Run diff -u file1.txt file2.txt for a Unified Output, which is the standard format used by developers (using + and - signs).

sdiff (Side-by-Side differences)

If you find standard diff outputs hard to read, sdiff displays the two files in a clean, side-by-side vertical split layout.

Command: sdiff file1.txt file2.txt
Output:

```
yoges@LAPTOP-DCISH7AJ:~/linux-fundamentals/comparing_files$ sdiff file1.txt file2.txt
apple                                                           apple
banana                                                        | grape
cherry                                                          cherry
orange                                                        | peach
```

- Takeaway: The pipe symbol (|) indicates that the lines are different. If a line exists in only one file, it will show a < or > arrow pointing to the file that owns it.

wdiff (Word-based differences)

*(Note: standard Ubuntu systems usually don't have vidiff installed by default, but they use **wdiff** for word-by-word diffs, or **vimdiff** for visual terminal editing. Let's look at wdiff and vimdiff).*

Command (Word diff): wdiff file1.txt file2.txt
Output:

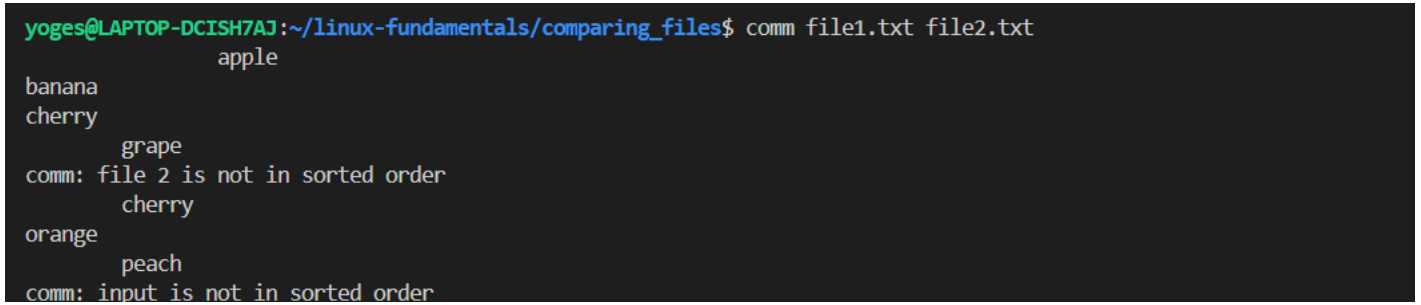
```
yoges@LAPTOP-DCISH7AJ:~/linux-fundamentals/comparing_files$ wdiff file1.txt file2.txt
apple
[-banana-]
{+grape+}
cherry
[-orange-]
{+peach+}
```

- **Takeaway:** It highlights specific deleted words inside [- -] and added words inside {+ +}.
- 💡 **Visual Terminal Alternative:** Run vimdiff file1.txt file2.txt to lock both files together in a color-coded Vim text editor split-screen. (Type :qa to exit).

comm (Compare sort files)

The **comm** command requires both files to be alphabetically sorted beforehand. It outputs data into three distinct columns: Column 1 (unique to File 1), Column 2 (unique to File 2), and Column 3 (lines common to both).

Command (Word diff): comm file1.txt file2.txt



- Takeaway: Because our sample files weren't fully sorted, the raw output looks a bit messy.
- 💡 Pro-tip: You can suppress columns using numbers. For example, comm -12 file1.txt file2.txt suppresses columns 1 and 2, showing only the lines that match exactly in both files (apple and cherry).

Option 1: Sort them on the fly (Recommended)

You don't need to manually rewrite your files. You can use Process Substitution <(sort file) to sort them temporarily in memory right as you run the command.

- Column 1 (Far left): Unique to file1.txt (banana, orange)
- Column 2 (Middle): Unique to file2.txt (grape, peach)
- Column 3 (Right indented): Found in both files (apple, cherry)



comm strictly requires files to be alphabetically sorted before it can compare them properly.

Option 2: Find only matching lines (The fix for your second command)

To get your comm -12 command to work perfectly and show only the common lines without any sorting errors, pipe the sorted files into it like this:

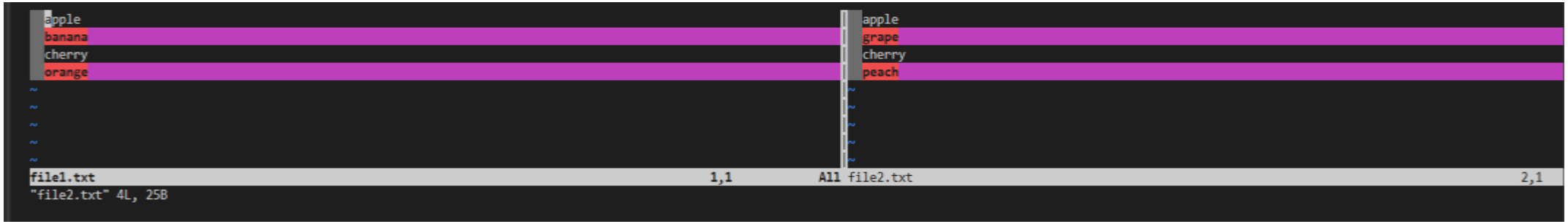
```
yoges@LAPTOP-DCISH7AJ:~/linux-fundamentals/comparing_files$ comm -12 <(sort file1.txt) <(sort file2.txt)
apple
cherry
```

Now that the sorting issue is solved.

vimdiff

To compare files using vimdiff, you can open multiple files side-by-side in the terminal to view, navigate, and merge their differences.

Command (Word diff): vimdiff file1.txt file2.txt
Output:



Vimdiff is a Linux command that can edit two, three, or four versions of a file with **Vim** and show their differences.